
QUAID-E-AWAM UNIVERSITY OF ENGINEERING SCIENCE & TECHNOLOGY

Python Programming Bootcamp - Week 4, Day 01

Introduction to File Handling

Atta Muhammad

attapanhyar@quest.edu.pk

+923331971110

Quaid-e-Awam University of Engineering, Science and Technology

Learning Objectives

- Understand the basics of file handling in Python.
- Learn how to open, read, write, and close files.
- Understand different modes of file handling (read, write, append).
- Handle exceptions while working with files

Why Learn File Handling?

- **Definition**: File handling is essential for data storage, processing, and analysis.

- **Applications**: Saving user data, logs, reading configurations, and exporting results.

Opening and Closing Files

- **Opening a File**

```
file = open("example.txt", "r")
```

- **Closing a file**

```
file.close()
```

•**Append Mode ('a')**: Opens a file for appending new content.
•**Binary Mode ('b')**: Used for binary files (e.g., images).

File Modes in Python

- **Read Mode ('r'):** Opens a file for reading.
- **Write Mode ('w'):** Opens a file for writing (overwrites content).
- **Append Mode ('a'):** Opens a file for appending new content.
- **Binary Mode ('b'):** Used for binary files (e.g., images).

Reading Files

- **Reading the Entire File:**

```
file = open("example.txt", "r")
content = file.read()
print(content)
file.close()
```

- **Reading Line by Line:**

```
file = open("example.txt", "r")
for line in file:
    print(line, end="")
file.close()
```

Writing to Files

- **Writing Data to a File**:

```
file = open("example.txt", "w")
file.write("This is a new line.")
file.close()
```

- **Appending Data to a file**

```
file = open("example.txt", "a")
file.write("\nAdding another line.")
file.close()
```

Hands-On Exercise 1

- Task: Write a Python program that reads the content of a file named **students.txt** and prints each line.

Using with Statement

- **Advantage**: Automatically handles file closing, even in case of exceptions.

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
```

Handling File Exceptions

- Catching Errors:

```
try:
    file = open("non_existent_file.txt", "r")
except FileNotFoundError:
    print("File not found!")
```

File Pointer and tell() & seek() Methods

- **Understanding the File Pointer**: Tracks the current position in the file.

```
position = file.tell()
file.seek(0)
```

Hands-On Exercise 2

Task: Write a Python program that writes user input to a file and then reads it back.

Reading and Writing Binary Files

- **Binary Files**: Used to handle non-text files such as images, videos, or executables.

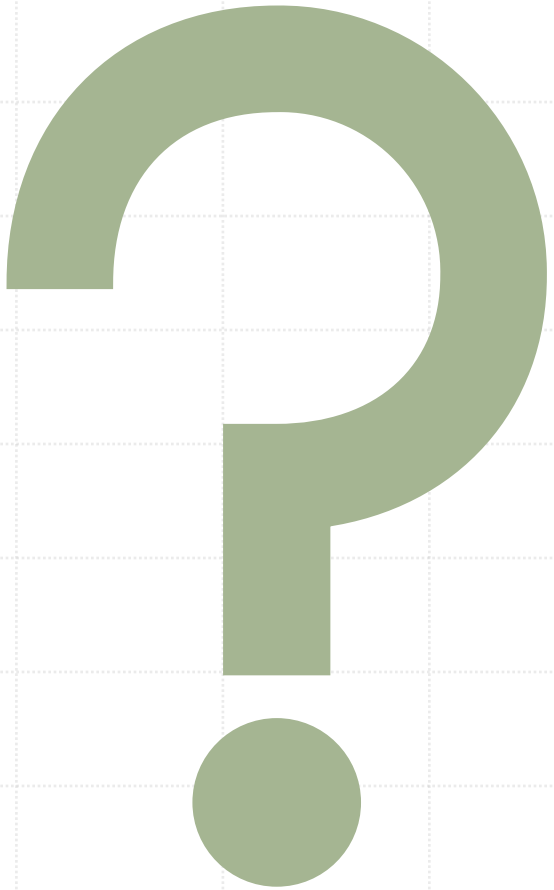
```
with open("QuestLogo.png", "rb") as file:
    data = file.read()
```

Hands-On Group Activity

- **Task**: Create a program that reads from a file data.txt and writes the reversed content to a new file reversed.txt

Recap and Key Takeaways

- Files can be opened in different modes (r, w, a, b).
- Always close files or use the with statement to handle them automatically.
- Exception handling is important to manage errors during file operations.



Questions ?

MATCH THE SOLUTIONS 😔

EXERCISE 😐

```
# Step 1: Open the file in read mode
with open("students.txt", "r") as file:
    # Step 2: Loop through each line in the file
    for line in file:
        # Step 3: Print each line (end="" removes extra newlines)
        print(line, end="")
```

Exercise 

```python
# Step 1: Open the file in read mode
with open("data.txt", "r") as file:
    # Step 2: Read the file content
    content = file.read()

# Step 3: Reverse the content
reversed_content = content[::-1]

# Step 4: Write the reversed content to a new file
with open("reversed.txt", "w") as reversed_file:
    reversed_file.write(reversed_content)

print("Reversed content has been written to 'reversed.txt'")
```

Exercise

```python
# Step 1: Get user input
user_input = input("Enter a sentence to write to the file: ")

# Step 2: Open the file in write mode and write the user input
with open("output.txt", "w") as file:
    file.write(user_input)

# Step 3: Open the file in read mode and read the content
with open("output.txt", "r") as file:
    content = file.read()

# Step 4: Print the content that was written to the file
print("Content of the file:")
print(content)
```